

```
#include <stdio.h>
```

```
int main() {  
int i, j, k, n = 3; // n=3 represents our number of equations  
float matrix[3][4], ratio, x[3];
```

```
// Matrix input (Values from your sample problem)
```

```
// Row 1:  $2x + 1y - 1z = 8$ 
```

```
// Row 2:  $-3x - 1y + 2z = -11$ 
```

```
// Row 3:  $-2x + 1y + 2z = -3$ 
```

```
float data[3][4] = {
```

```
{2, 1, -1, 8},
```

```
{-3, -1, 2, -11},
```

```
{-2, 1, 2, -3}
```

```
};
```

```
// Copying data from the input array to the working matrix
```

```
for(i=0; i<n; i++) {
```

```
for(j=0; j<n+1; j++) {
```

```
matrix[i][j] = data[i][j];
```

```
}
```

```
}
```

```
/* --- Step 1: Forward Elimination ---
```

```
Transforming the matrix into Upper Triangular form */
```

```
for(i=0; i<n-1; i++) {
```

```
for(j=i+1; j<n; j++) {
```

```
// Calculating the ratio to eliminate elements below the pivot
```

```
ratio = matrix[j][i] / matrix[i][i];
```

```
for(k=0; k<n+1; k++) {
```

```
// Row operation: Current Row = Current Row - (ratio * Pivot Row)
```

```
matrix[j][k] = matrix[j][k] - ratio * matrix[i][k];
```

```
}
```

```
}
```

```
}
```

```
/* --- Step 2: Back Substitution ---
```

```
Solving for variables starting from the bottom to the top */
```

```
x[n-1] = matrix[n-1][n] / matrix[n-1][n-1]; // Solve for the last variable (z)
```

```
for(i=n-2; i>=0; i--) {
```

```
x[i] = matrix[i][n];
```

```
for(j=i+1; j<n; j++) {
```

```
// Subtracting the known variable values from the current row
```

```
x[i] = x[i] - matrix[i][j] * x[j];
```

```
}
```

```
// Final division to find the variable value
```

```
x[i] = x[i] / matrix[i][i];
```

```
}
```

```
// Printing the final results
```

```
printf("\nSolution by Gauss Elimination:\n");
```

```
printf("x = %.2f\n", x[0]);
```

```
printf("y = %.2f\n", x[1]);  
printf("z = %.2f\n", x[2]);
```

```
return 0;  
}
```

output

Solution by Gauss Elimination:

x = 2.00
y = 3.00
z = -1.00